MVNOCoreSim: A Digital Twin for Virtualized IoT-Centric Mobile Core Networks

Stefan Geißler, Florian Wamser, Wolfgang Bauer, Steffen Gebert, Samuel Kounev, *Member, IEEE*, and Tobias Hoßfeld, *Senior Member, IEEE*

***Abstract*—The rapid growth of connected devices has led to the implementation of Internet of Things (IoT)-centric platforms designed to provide connectivity for machine-to-machine-type communication. Specifically, mobile virtual network operators (MVNOs) provide global service for IoT use cases by leveraging roaming in readily deployed physical networks. This global deployment of connected devices poses a significant challenge regarding the operation of centralized core networks with respect to dimensioning, scaling, as well as system survivability in case of signaling incidents. Overload control mechanisms for IoT mobile networks offer a proactive solution for mitigating excessive signaling traffic from IoT devices in mobile core networks. However, global scaling and a heterogeneous composition of devices present network operators with additional challenges. To address these challenges, this work presents a detailed, protocol-level simulation framework of a real-world IoT MVNO core network. We present both models for the expected IoT signaling load and the processing of signaling messages based on measurements in a live, production environment as well as a dedicated testbed. Finally, we present a case study on various overload mechanisms and identify critical performance characteristics to compare their performance. The results of this study categorize overload control mechanisms and shed light on the necessity for MVNOs to deal with the upcoming IoT traffic by defining appropriate overload control mechanisms in mobile core networks to ensure network survivability under unforeseen conditions.**

***Index Terms*—GPRS tunneling protocol (GTP), Internet of Things (IoT), measurement, mobile networks, network survivability, overload control, signaling, signaling transport (SIGTRAN), simulation.**

I. Introduction

THE RAPID adoption of the Internet of Things (IoT) in both industry and everyday life is leading to an ever-increasing number of connected devices. Network operators and providers react to this development with the ratification and implementation of machine-to-machine (M2M) IoT platforms that, in connection with mobile networks, enable global and demand-oriented access of IoT devices to the Internet [1]. Providers invest considerable efforts toward the transition from conventional communication to machine-controlled mass device connectivity in order to support the enormous number of devices, open up new profitable business cases, and give companies more configuration options and more flexible use of the mobile networks.

While initial forecasts regarding the number of devices have been overestimating the expected growth [2], more recent predictions expect up to 18 billion deployed IoT devices within the next few years [3]. Of these globally deployed devices, approximately 10% are expected to rely on mobile connectivity for data transmissions [4]. The volatility in the number of devices and tight economic reasons are encouraging providers and operators to take steps to operate IoT networks efficiently. The ability to mitigate short-term overload and general flexibility, e.g., during booms and downturns with millions of devices, has therefore become an indispensable measure and must be ensured by suitable overload control mechanisms [5]. This aspect specifically, and a general understanding of the scalability, performance, and resilience of these IoT platforms in general, are still the subject of active research [1], [6], [7]. In practice, signaling incidents occur [8] especially in the context of mobile virtual network operators (MVNOs) offering global coverage based on roaming agreements. Incidents like outages in (parts of the) network together with retrial behavior of IoT devices cause catastrophic signaling (over-)load in the core network. Therefore, appropriate overload control mechanisms need to be developed for mitigating excessive signaling traffic for this new, and not fully understood IoT traffic.

However, operators face the problem that they cannot simply modify or add new overload mechanisms in their deployed and running infrastructure. Small testbed replicas of the network infrastructure do not allow investigating scalability and overload situations properly. The concept of the digital twin [9], however, enables the evaluation of new mechanisms by developing a close, digital replica of any real world system before deploying them in the real world. However, there are currently no simulators available which are on the one hand abstract enough to allow simulation scenarios targeting at scalability of millions of IoT devices. On the other hand, the simulator needs to be detailed enough to take into account the expected signaling load and the processing of signaling messages in the core network of the mobile operator. To this end, researchers and operators require access to real networks to

Manuscript received 23 March 2022; revised 29 July 2022, 19 October 2022, and 16 January 2023; accepted 27 March 2023. Date of publication 30 March 2023; date of current version 25 July 2023. This work was supported by the Bavarian Ministry of Economics, Regional Development and Energy within the Project *5CALE* as part of the Research and Development Program for Information and Communication Technology in the field of 5G Mobile Communications. *(Corresponding author: Stefan Geißler.)*

Stefan Geißler, Florian Wamser, Samuel Kounev, and Tobias Hoßfeld are with the Department of Computer Science, University of Würzburg, 97070 Würzburg, Germany (e-mail: stefan.geissler@uni-wuerzburg.de; florian.wamser@uni-wuerzburg.de; samuel.kounev@uni-wuerzburg.de; tobias.hossfeld@uni-wuerzburg.de).

Wolfgang Bauer and Steffen Gebert are with EMnify GmbH, 97074 Würzburg, Germany (e-mail: wolfgang.bauer@emnify.com; steffen.gebert@emnify.com).

Digital Object Identifier 10.1109/JIOT.2023.3263258

2327-4662 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.

validate such simulation models. Due to the cooperation with a global MVNO, we are in the unique position to have access to real-world data for developing and validating such a digital twin, that has been developed and validated for a production network. Therefore, we are able to provide the world's first protocol-level simulation of a real-world MVNO core network in the context of IoT traffic, which we use to analyze scalability and overload mechanisms based on real-world incidents [1], [6], [8].

To this end, Section II formulates the problem we are addressing in this article and highlights the key contribution. Section III provides a summary of related work. Following, Section IV introduces the real-world mobile core system, while Section V introduces the simulation model and discusses necessary abstractions. A validation of the proposed simulation model is presented in Section VI. Section VII presents the case study and introduces the investigated overload mechanisms before providing results regarding their performance. Finally, Section VIII concludes this article.

II. Problem Formulation and Contribution

Recent studies show that IoT devices differ significantly from regular smartphone-based traffic both regarding their signaling and their payload transfer behavior [10], [11], [12], [13], [14]. Thus, networks in general and the deployed overload control mechanisms in particular must be able to mitigate short-term load peaks, excessive signaling traffic, or abnormal behavior of machines. Characteristics of machine-type communication become particularly problematic under unexpected circumstances, e.g., in the event of regional or global network failures, connection resets, or general loss of connection. In such scenarios, the reaction of devices is entirely dictated by their programming, firmware or configuration. In many cases, as opposed to humans, devices that lose connectivity will continuously try to re-establish connectivity, thereby triggering significant changes regarding the workload in the control plane.

A. Problem Formulation

It becomes even more evident when taking into account recent developments in the area of MVNOs that run centralized instances of the mobile core without deploying their own radio access network (RAN). Instead, MVNOs rely on roaming agreements with physical mobile network operators (MNOs), thereby exploiting existing infrastructure in combination with the global IP exchange network (IPX) [15] to carry signaling traffic between visited networks and the centralized mobile core of the MVNO. This setup allows a single MVNO to operate a single, centralized instance of the mobile core while still providing global connectivity to its customers. However, this also introduces multiple points of failure for both MNOs and MVNOs that may lead to extreme overload scenarios. Being dependent on the infrastructure of many, globally distributed physical operators as well as a number of signaling carriers to ensure transmission of signaling traffic between the visited and home network introduces the risk of regional outages resulting in signaling storms [16] that can quickly overload the home network core. We answer the following questions: where is the bottleneck in the core network of an MVNO for IoT traffic? What are appropriate resource scaling and overload mechanisms?

However, an operator cannot simply test such mechanisms in the real-world deployed system used by its customer. Instead, mechanisms must be tested beforehand regarding their performance and efficiency. Due to the issue of scalability and overload, large-scale testbeds are required to test for realistic scenarios in terms of number of devices and emerging traffic. However, this would require a duplication of the entire MVNO network, causing large costs. Simulations are an appropriate tool to overcome this issue. The problem we are addressing is the following: what is the right abstraction degree of the MVNO core network simulation? How to model the signaling traffic emerging from real-world device behavior? Is the simulation model valid?

B. Contribution of This Work

As a key contribution of this work, we introduce a detailed, protocol-level simulation framework developed and validated on the base of a real-world, virtualized MVNO core network. The proposed simulation model contains, on the one hand, an abstract signaling model dictating signaling message sequences as well as timeout and retry behavior based on the interaction between IoT devices and visited as well as home network control plane components. This abstract model is based on observations made in the real system during recovery after failure and represents the messages exchanged between components in the visited network, such as the visitor location register (VLR) or serving GPRS support node (SGSN), and the home network. On the other hand, we present a detailed model of a 2G/3G MVNO core network describing the processing of received signaling messages using network and component simulations with processing times obtained from a commercial MVNO mobile core network that processes IoT traffic from 540 RANs with IoT devices in 180 countries. In this article, we focus on the 2G/3G architecture, as shown in Fig. 2, and the associated signaling traffic as this is by far the dominant type of signaling currently originating from IoT devices [1].

Furthermore, we present a case study of how the proposed simulation framework can be applied to perform bottleneck detection and investigate the impact of resource scaling within the core network. Finally, we present a detailed investigation of various overload control mechanisms that allow the continuous survival of the core network under extreme overload conditions. We identify several performance metrics beyond a mechanism's mitigation capability and compare the suggested mechanisms regarding, e.g., resource utilization and blocking probability from device point of view. The results of this article help categorize overload control mechanisms and highlight the need for MVNOs to deal with the upcoming IoT traffic by defining appropriate control mechanisms in mobile cores. Furthermore, to the best of our knowledge, this is the first protocol-level simulation of a real-world MVNO core network in the context of IoT traffic.

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.

In summary, the key contributions are as follows.

1) Detailed, a protocol-level simulation framework of a real-world MVNO core network which is made available to the research community as open-source implementation.
2) Signaling model for the expected signaling load with the arrival rate as its singular configuration parameter.
3) Digital twin representation of core components involved in processing signaling messages including their service time distributions as obtained via measurements in a live, production environment.
4) Validation of the proposed models.
5) Case study on dimensioning: bottleneck detection and resource scaling.
6) Development of a novel overload control mechanism ("device policing") dedicated for MVNO core network.

III. Related Work

When it comes to research related to the contributions made in this work, we distinguish two different research areas. On the one hand, we provide a summary of recent investigations into mobile network architectures and their performance. On the other hand, we dissect literature regarding overload control as well as network survivability, with a focus on mobile and other radio networks.

A. Mobile Network Architectures

Several works have been conducted investigating various aspects of mobile core networks and MVNOs in particular.

Bedhiaf et al. [17] investigated different MVNO architectures, with respect to (wrt.) their deployment time and overall suitability for virtualization. Li et al. [18] performed an extensive, crowdsourced measurement study and investigate aspects of inaccurate billing and performance discrimination by examining user plane traffic flows as well as several external data sources.

When it comes to the simulation of mobile networks, previous research has looked into simulating radio access systems and signal propagation [19], [20]. Metzger et al. [21] investigated the performance of a virtual GGSN. Samoilenko et al. [7] investigated the impact of the presence of IoT traffic on human communication in LTE environments. However, the simulation proposed in this work is, at the time of writing and to the best of our knowledge, the first protocol-level simulation of a mobile core architecture in the context of IoT.

Finally, there are also some works on the modeling and characterization of IoT traffic. Mazhar and Shafiq [22] focus on smartphone or human generated traffic. In [23], the emerging trends of network traffic classification in IoT and the utilization of traffic classification in its applications are presented.

B. Overload Control

Overload or congestion control date back to the beginning of radio networks. Originally, overload control was done at the air interface or RAN of mobile networks with radio resource management [23], [24], [25], [26], [27]. The challenge here was to efficiently assign subscribers and devices to radio resources of a cell. However, with the introduction of smaller or even cell-free architectures for machine-type communications as well as the new role of virtual operators, this went out of the focus in favor of signaling overload control in the mobile core [28], [29].

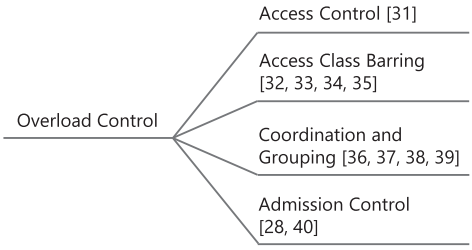


Fig. 1. Categorization of overload mechanisms applicable to core networks.

Common overload mechanisms for core networks are simple dropping or probabilistic dropping [28]. More advanced overload mechanisms are specifically designed to deal with bursty IoT traffic [28] or leverage explicit signaling [29]. A detailed dissection of overload and congestion control mechanisms for both mobile core and RANs is provided by Ferdouse et al. [30]. There, core network access control mechanisms can be roughly categorized into four groups, as summarized in Fig. 1.

First, *Access Control* deals with overload by barring devices directly within the access network, e.g., at the evolved NodeB (eNB), or at early hops in the network, such as the mobility management entity (MME) or SGSN [31]. However, as in the scenario evaluated in this work the MVNO has no control over the RAN or the visited network core components, these mechanisms are not applicable.

Second, access class barring (ACB) attempts to alleviate overload situations by proactively assigning different access classes to devices that are subsequently denied from attaching by the eNB, based on their class and barring factor. Several extensions of the original ACB mechanism have been proposed in the past [32], [33], [34], [35]. This class of mechanisms again assumes direct interaction with devices in the field. Even more, IoT devices are assumed to support the different barring mechanisms proposed in literature. This, however, is unlikely in real deployments, as devices are manufactured as cheaply as possible, and hence only provide the most basic features.

The class of *Coordination and Grouping* mechanisms attempts to resolve overload scenarios by introducing features to devices and eNBs that allow the explicit coordination [36], [37], [38], [39]. Again, interaction between devices and the radio network or visited network is required, making this class of mechanisms ill-suited for the overload problem in MVNO environments.

Finally, *Admission Control* mechanisms attempt to combat overload by blocking specific devices, similar to the above-mentioned ACB mechanism [28], [40]. The difference here is that additional congestion information is available to make more efficient decisions, as the decision is not made by the eNB. Instead, a function much deeper in the core is making the decision here, e.g., the Serving Gateway (S-GW) or Packet Data Network Gateway (P-GW).

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.

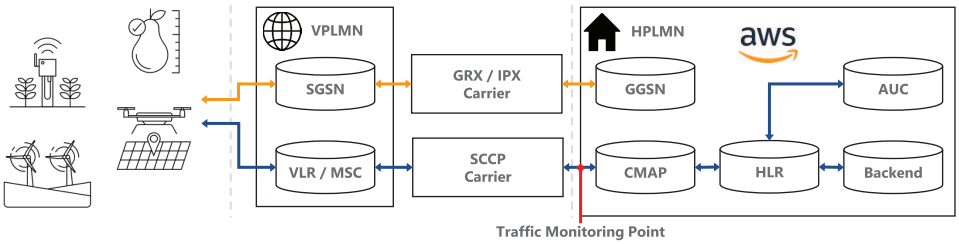


Fig. 2. Schematic overview of the technical system used as the basis for the simulation model.

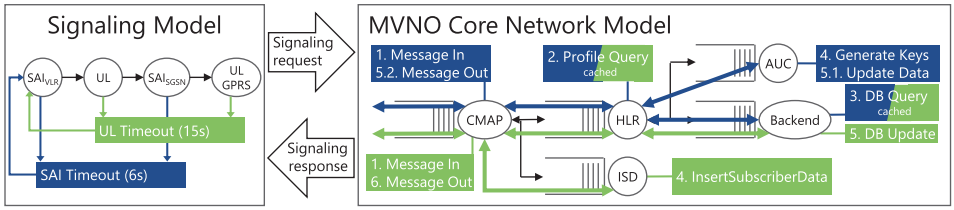


Fig. 3. Schematic overview of the simulated system consisting of the signaling model and the MVNO core model.

Based on these works, we distill critical properties from multiple approaches and design and compare multiple overload control mechanisms that can be applied to the MVNO core networks, as no interaction with the visited network or even end devices is required. According to the categorization above, the mechanisms developed in this work belong to the class of admission control mechanisms.

IV. Technical System Description

As the base for the simulation model, we use a state-of-the-art virtual mobile core implementation that is, at the time of writing, in production and serving upward of 170 000 unique devices and 21 million signaling dialogs on a daily basis. In the following, we present the key technical aspects of this system and highlight central properties relevant for the development of our simulation model.

The system is designed from the ground up to operate in the cloud, adhering to the cloud native paradigm [41], [42] and the principles of network functions virtualization (NFV). All involved services are implemented in software and running on Amazon Web Services (AWSs). Besides advantages of fine-grained scalability, this dramatically lowers efforts compared to hardware development and enables in-house development of unique selling points without dependency on vendors. By leveraging multiple AWS Regions, a globally distributed, multilayered control plane for handling GPRS tunneling protocol (GTP) messages has been developed [43] and is in production since 2015.

Leveraging the AWS public cloud ecosystem, enables the mobile core system to operate globally and scale virtually limitless to meet the required capacity. The latter, as well as resiliency against component failure, however, require more than just the underlying cloud architecture can provide.

Instead, paradigms from the area of serverless computing [44] are used to allow dynamic scaling, optimize resource utilization and allow quick recovery after component failure. More specifically, all components of the architecture are developed as an actor-based model [45], [46] on top of Akka [47]. The advantage of the actor model is a simplification of applications handling a high number of concurrent tasks, in contrast to object-oriented programming that often requires locking of data structures. While actors communicate via immutable messages, every actor processes only one message at a time. Until a message can be processed, it waits in the actor's mailbox. Communication across multiple hosts is transparently and resiliently given by the Akka Cluster extensions.

Fig. 2 shows a schematic overview of the full system and its environment, starting with IoT devices deployed in the field on the left, the visited network used for roaming to establish mobile connectivity and the corresponding signaling carriers in the middle, and the virtual core network on the right. Note that only services relevant for this work are shown in the image. The path taken by GTP control (GTP-C) signaling messages is shown in orange, the blue path indicates the path taken by signaling transport (SIGTRAN) signaling messages. Finally, the point at which externally arriving traffic is monitored is shown by the red indicator. The devices on the left as well as the signaling carrier part in the middle are not the focus of this work and in the following subsumed in our signaling model (cf. Fig. 3). The core network model focuses on the right part of Fig. 2, representing the virtual mobile core platform as well as its components. Note that this work focuses only on the SIGTRAN processing path shown in blue, as well as the four components involved therein. The CMAP component, named after the camel address part (CAP) and mobile application part (MAP) protocols, presents the ingress and egress nodes for both signaling requests and responses, respectively. It receives messages and translates them into the data structure used by the subsequent components before forwarding the request to the next component in the chain. The home location register (HLR) is, as specified in [48], involved in keeping the state of mobile subscribers and handles signaling interactions related to, e.g., authentication or updating the location of devices. To this end, the HLR delegates tasks to the two remaining components. The authentication center (AUC) is thereby responsible for the authentication of devices. The Backend represents the

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.

interface toward persistent storage and is involved in both querying and updating device profiles. Finally, it is important to note that both the HLR and the Backend component feature caching capabilities that allow requests to be processed directly without the need for task delegation. Thereby, the HLR caches device profiles for 20 min while the Backend cache removes entries after 24 h. The cache size is thereby only limited by the available system memory, and is hence assumed to be infinite for the remainder of this work.

Each of these components is then realized by reserving a specific number of CPU cores dedicated for the specific component. This resource reservation for individual components ensures that all tasks can be performed at any time without the need for intertask scheduling. However, within each of the four core network components, the actor-based programming model realized by the Akka framework creates a serverless pipeline. This means all resources available within each component's pool can be freely distributed among all processing tasks of the specific component.

The implications for the developed simulation and the resulting abstractions inferred from these system properties are detailed in the following section.

V. Simulation Model Description

After detailing the technical characteristics of the real-world system used as a basis for the developed model, this section presents a description of the simulation model. Fig. 3 shows a schematic representation of the system. The simulation consists of two core components, namely, the *signaling model* dictating the workload resulting from the interaction between IoT devices and their visited network, as well as the interaction between visited and home network. This abstract model subsumes all elements placed left of the traffic monitoring point in Fig. 2. The *MVNO core model* encompasses the components in the virtual mobile core dealing with the processing of signaling messages. Both models are based on preliminary studies of a production-ready state-of-the-art virtual core network for IoT devices and applications. The code for all simulation scenarios presented in this work is publicly available on GitHub.[1]

A. Signaling Model

The signaling model represents an abstract representation of the signaling traffic observed between visited and home network resulting from roaming IoT devices. It dictates characteristics such as the sequence of signaling interactions, as well as retry and timeout behavior. Note here that due to the technical properties of the real-world system, our signaling model does not actually model signaling messages used by IoT devices, but rather the messages triggered by the VLR and SGSN of the visited network that are then issued toward the mobile core network, as is the case during roaming.

In order to keep the number of parameters of the simulations conducted in this work in check, we define a singular signaling model that is used for all simulation scenarios throughout this work. The model used is a representation of a scenario in which devices attach to the system, assuming the system holds no prior information about the respective devices. This model has been chosen as it reflects the behavior that occurs after a system outage that leads to loss of connectivity and the subsequent attempt to reattach to the mobile network, as well as completely new devices that connect to the system for the first time. Note that in the following, we refer to devices not as physical IoT devices, but abstract objects generating signaling load.

The signaling model used in this work exhibits a single configuration parameter, namely, the number of new arrivals per second. As we have previously shown in [6], it is feasible to assume an infinite number of sources and, based on Palm-Khintchine, negative exponentially distributed interarrival times. Hence, the only parameter required to configure the signaling model is the arrival rate of new devices. After arrival, each device performs a specific process consisting of four steps, that need to be processed successfully and in order. Only after successful completion of all four steps shown on the left-hand side of Fig. 3 a device is considered attached and leaves the system.

Specifically, the VLR starts by issuing a sendAuthenticationInfo (SAI) message request used for authentication at the core network. Based on the origin of this message, we call this first message SAI_{VLR}. Upon success, an updateLocation (UL) message is issued to update information about the device, such as the visited operator and location, in the HLR database. At that point, the device would be able to make calls and transmit text messages. However, since a majority of devices uses data connectivity [6], we assume all interactions also perform the equivalent attach procedure for GPRS connectivity consisting of another SAI message, called SAI_{SGSN}, followed by an *updateGprsLocation* message (UL_{GPRS}). After successful completion of this sequence of messages, we consider the attachment procedure complete and the corresponding element leaves the simulation.

In addition to the message sequence, the signaling model defines two separate timeout paths, indicated in blue and green in the signaling model part of Fig. 3. The green path shows the timeout started after an UL message is issued. If the respective interaction is not completed within 15 s, the system assumes loss of connection and returns to the beginning of the signaling sequence. Analogously, the same occurs after not receiving a response to a SAI message for 6 s. The retry values for both UL and SAI messages have been provided by the MVNO and are based on expert knowledge as well as values observed in the underlying data set which was used to develop the signaling model. Note that, due to confidentiality reasons, the raw data used to obtain these values is not included in the supplemental GitHub repository.

Naturally, and as already mentioned, the signaling model used to obtain the simulation results presented in the next sections is only an abstract representation of how the signaling traffic observed at this point in the system behaves in reality. However, the developed simulation tool is explicitly not limited to such abstract models but can also incorporate much more detailed models as well as a combination of various different models to simulate different signaling patterns.

[1] https://github.com/lsinfo3/MVNOCoreSim

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.

TABLE I
MVNO CORE MODEL COMPONENTS AND PARAMETERS

Step	Name	Component	Description	Capacity	Processing Time [µs] Min	Mean	Max	Coeff. of Var.
1	Message In	CMAP	Parsing messages	8 Processors	1209	1659	2245	0.07
2	Profile Query	HLR	Get device profile	64 Processors	1282	1739	2357	0.06
3	DB Query	Backend	Get device profile from DB	128 Processors	671, 38010	1877, 70482	17302, 2332033	0.06, 0.68
4	Insert Subscriber Data	ISD	Outgoing interaction with visited network	∞ Processors	1969	5998	204101	1.43
5	DB Update	Backend	Update database content	128 Processors	11239	13115	55738	0.25
4	Generate Keys	AUC	Generate keys for authentication	8 Processors	2487	2886	3485	0.04
5.1	Update Data	AUC	Update device data in DB	8 Processors	4357	4960	5401	0.03
6, 5.2	Message Out	CMAP	Transmission of response	8 Processors	299	749	1368	0.14

Especially taking into account the findings in [6], this combination of models will be required to obtain workloads that match reality.

B. MVNO Core Model

The second component of the simulation is the MVNO core model that dictates the interaction between services within the mobile core, i.e., the path messages take through the system, as well as their capacities and processing performance. Furthermore, this component will be responsible for implementing any specific system extensions or mechanisms, as we will demonstrate later in Section VII. The right side of Fig. 3 depicts the simulated core components. The annotations indicate the processing steps performed during the processing of SAI (blue) and UL (green) messages. Note that the processing paths for both SAI_{VLR} and SAI_{SGSN} as well as UL and UL_{GPRS} messages are, aside from differences due to caching, identical.

In general, the MVNO core model is defined by a network of queuing components, their capacities, and processing performance as well as the paths messages take through the queuing network. Table I lists the processes messages go through on their path through the core, as well as their key parameters. Note that the provided capacities are per component, and all steps performed by the same component share this capacity.

The capacities as well as the processing times used for these operations are modeled after the real-world system and have been obtained through dedicated measurements. Although only the mean processing time values are presented in Table I, the simulation input actually consists of a vector of processing time samples obtained from the real-world system. These samples are also included in the accompanying GitHub repository. Furthermore, all queues are assumed to hold an infinite number of elements, as the real-world system is only limited by memory capacity and never actively rejects messages without overload control or congestion mechanisms.

Based on these operations, we can now define the paths of the three signaling messages used in the signaling model. Note that the *Insert Subscriber Data* operation scales with an infinite number of processors. This is done as the processing for this outgoing interaction is performed by the visited network which, for simplicity reasons, is represented as a dummy component with infinite capacity, based on the assumption of similar response times independent of the number of requests the MVNO is issuing. This also means that events never have to wait before being processed by the ISD component.

As already detailed for the signaling model, *SAI* messages occur in two variants, SAI_{VLR} and SAI_{SGSN} which are processed mostly identical. The blue annotations in Fig. 3 depict the involved processing steps. Incoming messages are parsed by the CMAP component (1), routed to the HLR (2) where the actual message processing is performed. At this point, the difference between the two types occurs due to caching. Namely, in order to process the initial SAI_{VLR} message, the HLR needs to query data from the Backend component (3). Depending on the cache of the database, this step exhibits two different processing time distributions, hence we also show two mean values in Table I. Additionally, the response will be cached at the HLR for subsequent SAI messages belonging to this device, meaning subsequent messages may skip the DB query (3) entirely. Finally, the AUC generates the authentication keys (4) and performs the processing required for authentication (5.1). In parallel, a reply is sent via the CMAP back to the visited network (5.2). Note that steps (5.1) and (5.2) are performed in parallel.

When it comes to UL and UL_GPRS messages, the green annotations depict the involved processing steps. Here, it is important to note that, due to the signaling behavior employed in this work, all UL and UL_GPRS messages generate a cache hit at the HLR (2) and can hence skip the DB query (3). However, the DB Update (5) has to be performed either way.

In this work, both HLR and Backend caches exhibit infinite space and are configured to retain elements for 20 min in the case of the profile query operation and 24 h in the case of the DB query operation.

Finally, in order to keep the complexity of the core model in check, a number of abstractions over the real system have been made during the development of the simulation model. These mainly revolve around the representation of individual components. As is typical for cloud native architectures, the real system realizes each component as a group of dedicated instances, each equipped with a certain capacity, i.e., CPU cores. As an example, the AUC component is realized using four instances with two CPU cores each, for a total of eight processors. In this setup, dictated by implementation specifics of the services, each instance features its own queue. For the simulation model, we abstract this behavior and just implement a single instance with a single queue and the total

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.

number of processors instead. This abstraction will, based on the multiplexing gain in queuing systems, overestimate the performance of the real system.

Furthermore, we introduce an additional abstraction with respect to the processing path that is presented in Fig. 3. Namely, due to technical limitations, we were not able to obtain accurate processing time distributions for some steps along the processing path. More specifically, some processing steps take such little time, that their processing times are well below the margin of error of time measurements in software environments. Due to these inconsistencies and to avoid overfitting, the simulation model is omitting these processing steps. One example for this is the forwarding of the response of a SAI message by the HLR—between (4) and (5.2) in Fig. 3. This abstraction will, in general, lead to an underestimation of the response times for these message types. However, as the processing times for the omitted steps are negligible and are within the range of 100 ms, the effect on the overall results is expected to be negligible as well.

VI. Validation

Based on the simulation model introduced in the previous section, we now validate the model by comparing results to measurements taken from real systems. We compare the results of a dedicated simulation run with values obtained from a dedicated testing environment deployed in AWS provided by a real MVNO. Furthermore, more validation data was recorded during the productive use by the MVNO and compared with the values from the test system and the simulation.

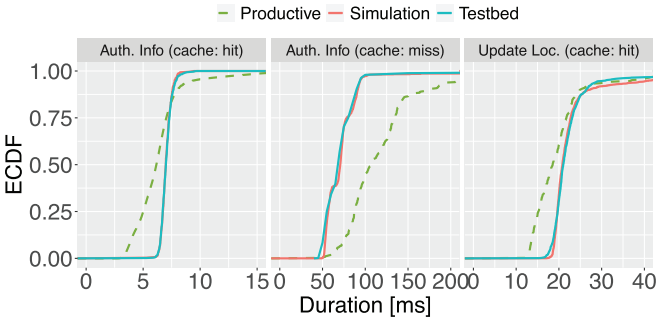


Fig. 4. Validation of processing times of dialogs in testbed and simulation; additionally, values of productive environment.

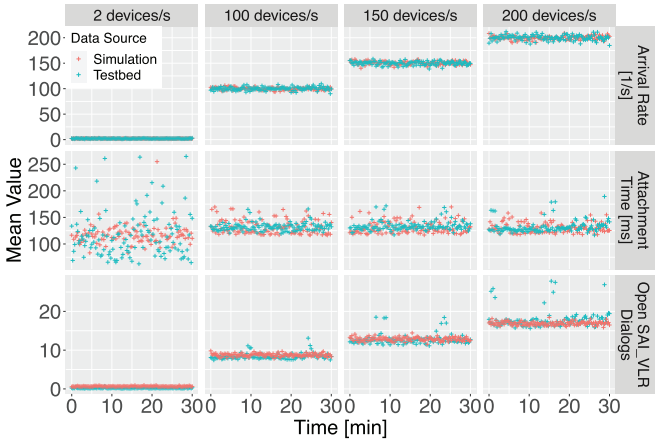


Fig. 5. Validation of configured arrival rate, attachment time and number of open SAI_{VLR} dialogs.

A. Simulation Accuracy

We start by validating the processing time values reported by the simulation against both measurements from the testing and the production environment. The results of the productive data set have been collected in November 2020. The data set was sampled from all customers from 193 countries and consist of 1 601 442 messages and dialogs. After post-processing, 206 971 message flows could be isolated to reveal the processing times of components in the MVNO core network per dialog interaction. For the testbed measurements, the device behavior model was configured to generate new device arrivals with geometrically distributed interarrival times with a rate of two devices per second. Both the testing and the simulation environment as well as the production system use the capacities and processing values presented in Table I.

Fig. 4 shows the response times for both cache misses and cache hits for SAI messages, as well as cache hits for UL messages. It is important to note here that, due to the modeled signaling behavior and time scales, both *UL* and *updateGprsLocation* messages always result in a cache hit, as they require a successful SAI to be issued in the first place, which triggers a cache miss and hence fills the cache. Moreover, the SAI_{SGSN} message, cf., Fig. 3 also always results in a cache hit for the same reason.

The figures show the response time along the x-axis and the empirical CDF along the y-axis. All subplots show the values obtained from the testing environment in blue, from the productive one in green, and the distributions resulting from the simulation in red. The simulated values correspond to the CDFs of the testbed measurements for all measured dialogs. It can also be seen that the values from the productive system are of a similar order of magnitude, but have slightly different values due to the different load and configurations in Amazon AWS than in the pure testbed.

Furthermore, the Kolmogorov–Smirnov distance (KSD) and the Jensen–Shannon divergence (JSD) as distance metrics have been derived between the testbed and simulation distributions to better show that the CDFs match each other in numbers. For SAI_{VLR}, the one-sample KSD with the simulation samples is 0.097 with a p-value of 0.94, meaning there stands nothing against the hypothesis that the samples are drawn from the same distribution. The JSD is at 0.04 for SAI_{VLR}. For SAI_{SGSN} with cache hits, the KSD is at only 0.030 with a p-value of 0.99, the JSD assumes 0.004. The KSD for *UL* is at 0.064, JSD with 0.0251.

Fig. 5 shows three central KPIs and compares the results obtained via simulation to values observed in the testing environment. Thereby, horizontal facets indicate the configured arrival rate of new devices while vertical facets state the evaluated metric. Each facet then shows the mean metric value obtained for 15 s intervals over time for simulation and measurement runs of 30 min duration. Note that the simulation time has been set to 40 min and the first and last 5 min have

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.

TABLE II
MEAN AND 95% CONFIDENCE INTERVAL FOR MEASUREMENTS AND SIMULATION. COLORS INDICATE WHETHER CONFIDENCE INTERVALS DO OVERLAP

Arrival Rate		2	100	150	200
Att. Time	Mmt	117.41±3.51	131.99±2.41	130.97±2.02	130.53±1.68
	Sim	116.93±7.66	132.24±1.39	133.93±1.67	132.33±2.13
Open Dialogs	Mmt	0.65±0.01	8.76±0.06	12.86±0.08	16.91±0.08
	Sim	0.21±0.01	8.43±0.14	12.74±0.24	17.81±0.40

been trimmed to obtain stationary results. Similarly, the figure shows a 30-min snapshot of a longer measurement run from the testbed.

The first row shows simply the monitored arrival rate of new devices and shows that both the testbed and simulation have been subjected to a similar load between 2 and 200 arrivals per second.

More interesting, the second row shows the attachment time of devices, meaning the time it takes for each device to complete their attachment cycle as described in Section V-A. It can be seen that the simulation results closely align with the observed measurement values for arrival rates of 2, 100, 150 and 200 devices per second when it comes to the magnitude of values. The peaks observed in both the attachment time and the open SAI_{VLR} portion of the figure are explained by fluctuations of the testbed performance due to it running in a public cloud environment. More specifically, a closer investigation has shown that the database underlying the Backend component took longer to respond to queries. Unfortunately, due to the nature of the monitoring tool used in the testbed, it is unknown if single large outliers or a general increase in response time occur at these points in time. In order to not overfit this specific system, the simulation neglects these fluctuations. However, such changes in processing power can easily be reflected in the simulation by simply modeling the processing times of components as a function of time, if desired. In fact, even the dynamic adaptation of CPU capacities could be realized with minimal extension of the simulation. The fluctuations observed in the low load scenario with two arrivals per second is simply explained by the low number of samples in each 15-s interval.

Similarly, the last row of Fig. 5 shows the mean number of open SAI_{VLR} dialogs. A dialog is thereby considered as open as long as it is being processed by or waits for processing by any of the core components depicted in Fig. 3. Again, the figure shows a close match between simulation and measurement values for all load levels over the whole run.

Analogously, Table II shows the mean values and 95% confidence intervals for the attachment time as well as the number of open SAI_{VLR} dialogs for all evaluated load levels, corresponding to the second and last row of Fig. 5. The colors indicate whether the confidence intervals do overlap (green) or do not overlap (red). It can be seen that for the attachment time, all scenarios result in overlapping confidence intervals, highlighting the close match between simulation and measurements. When it comes to the number of open dialogs, the simulation does deviate from the measurement values, as the resulting confidence intervals only overlap for an arrival rate of 150 devices per second. However, when comparing the mean values, it can be seen that the values generated by the simulation closely approximate the results observed during measurement. On the one hand, the deviation for this metric can be explained by the outliers that can be seen in the bottom row of Fig. 5. On the other hand, the observed differences are well within the margin of error of the measurement setup hosted in a virtualized, distributed environment. Additionally, the simulation input consists of a set of empirical distributions, thereby neglecting any form of correlation and autocorrelation that will occur in practice. Hence, small deviations between the simulation and measurement values are to be expected.

Finally, based on these observations as well as the dialog-level response times evaluated in Fig. 4, we can conclude that the proposed simulation model provides a close representation of the real-world system used during the measurement runs when it comes to the KPIs reported by the simulation. However, it is critical to note that regarding the quality of the results for extreme load levels, and overload specifically, there is no reliable way to compare values obtained via extrapolated load levels from the simulation to the real world system. Specifically, the impact of the evaluated overload control mechanisms on the real-world system is hard to evaluate, as a real world implementation of the evaluated mechanisms does, at the time of writing, not exist. However, due to the way the system works and the way the overload control mechanisms have been designed, no modification of the processing behavior of the system itself is included in any of the mechanisms. This means, that the processing path, timing, and all other relevant characteristics of the system are identical, whether there is one arrival per second or fourfold overload. The only difference is the dropping of messages that would result in a timeout in any case, thereby reducing system efficiency. Obviously, there remains the question of the impact of higher fill levels of the queues at each processing component. However, the mechanisms are explicitly designed to minimize the impact of internal queuing. Hence, we argue that the extrapolation is a valid tool here to show the performance of the evaluated mechanisms under overload conditions.

B. Simulation Functionality

Furthermore, in order to show the correctness of the implementation of the device behavior model as well as the path of messages through the core on a functional level, Fig. 6 shows a waterfall plot of an exemplary device that represents the timeline of processing steps of this device along the x-axis. The y-axis shows the different signaling dialogs used by the device. The colored bars indicate the resource that is occupied by this specific device at the specific point in time. The vertical lines indicate special events such as cache hits and cache misses.

The figure shows the sequence of occupied components for each signaling message. For example, the SAI_{VLR} shown in the

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.

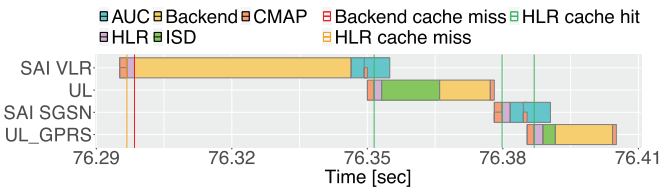


Fig. 6. Waterfall plot of an exemplary device and its signaling behavior.

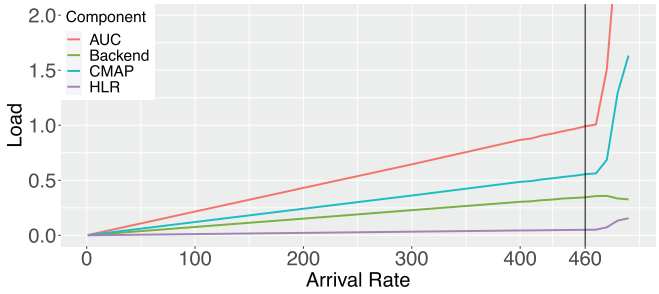


Fig. 7. Component load under increasing arrival rates of new devices.

first row arrives at the CMAP component, is then processed by the HLR, where a cache miss occurs. Based on that, a database query is triggered and the Backend component gathers the required information before the AUC starts generating the entries required for authentication. Finally, the CMAP component transmits the response while the AUC simultaneously updates database entries. Note the missing occurrence of the HLR between the Backend and the AUC, that is omitted in the simulation due to the reasons elaborated earlier in Section IV. Analogously, the other signaling interactions are shown in the further rows of the plot, each of which behaves as described in Fig. 3.

VII. Case Study—Dimensioning and Overload Control

After demonstrating the close fit between the real-world system and the simulation model, this section presents a case study evaluating different aspects of the system to show the capabilities of the developed simulation model. We start with a bottleneck detection and show the possibility of scaling individual core components. Following, we develop and compare different mechanisms that can be applied to mitigate system overload and ensure system survival under overload conditions.

A. Bottleneck Detection

In order to establish a baseline for the maximum load the system can handle without failing and in order to detect which component acts as the bottleneck, we perform simulation runs with a duration of 5 min for an increasing arrival rate of new devices. For these runs, we evaluate the mean load for each of the MVNO core components, as described earlier in Section V-B.

Fig. 7 plots the arrival rate of new devices in arrivals per second along the x-axis against the encountered load along the y-axis. The colors indicate the core services as introduced in

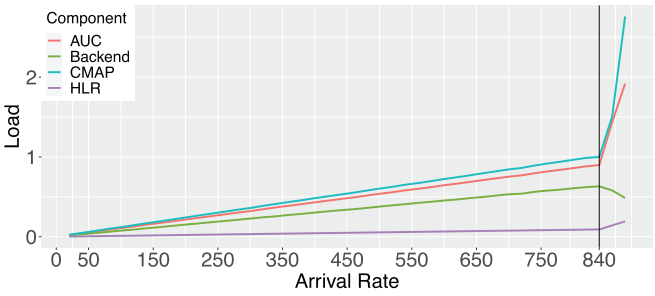


Fig. 8. Component load under increasing arrival rates of new devices with rescaled AUC.

Section V-B. We identify the load level at which the first core components exceeds its maximal processing capacity as the baseline for the following scenarios. Load is thereby defined as the ratio between the mean number of messages arriving at the system and the number of messages that could have theoretically been processed by the system per time unit. The number of messages that can be processed is thereby inferred based on the processing distribution. Hence, a mean service time of 10 ms would lead to a maximum number of 100 processed messages per second. This, in combination with a mean inter-arrival time of 20 ms (or 50 arrivals per second), would result in a load of 0.5. The vertical indicator line shows the highest arrival rate that leaves all components with a system load below 1. The component with the highest load in this case, and hence the bottleneck, is AUC with a load of 0.99. The figure also shows different behavior of components when increasing the arrival rate over the maximum value of 460 arrivals per second. This spike is explained by the device behavior introduced in Section V-A. As, due to overload, the queues of components start to fill and devices start to run into timeouts, resulting in an infinite retry loop. This leads to an additional increase in signaling messages over time, as more and more devices try, and fail, to attach to the system. Due to this behavior, the system enters a *livelock*, as all components work at maximum capacity, but due to the long waiting time, devices retry before the system can respond.[2]

At the same time, the Backend component exhibits a decline regarding its load. This is due to the fact that the Backend service is only involved in processing UL and UL_GPRS messages. However, since AUC is already overloaded, devices fail during authentication and the rate of UL messages drops.

B. Resource Scaling

After identifying the maximum rate the system can handle without running into overload at a rate of 460 new devices per second and identifying the AUC as being the bottleneck holding back the system, we now rescale the AUC component from 8 processors as shown in Table I, to twice the capacity with 16 processors and repeat the bottleneck detection regime for the rescaled system.

Fig. 8 again shows the load of individual core components. It can be seen that rescaling the AUC has shifted the bottleneck

[2]*Note:* This behavior can also be observed in the real production system if no measures are taken.

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.

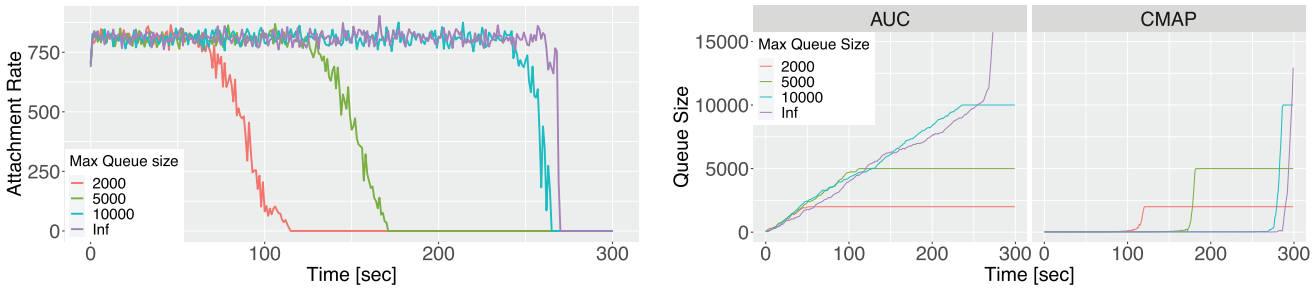


Fig. 9. Performance Metrics over time for different maximum queue sizes. (a) Attachment rate. (b) Queue size for AUC and CMAP components (*y*-axis limited to 15000).

TABLE III
SUMMARY OF OVERLOAD CONTROL MECHANISMS

Name	Queueing	Dropping	Prioritization	Explicit Signaling
Baseline	✔	✘	✘	✘
Simple Dropping	✔	✔	✘	✘
MSU Policing	✔	✔	✔	✘
Device Policing	✔	✔	✔	✔

to the CMAP component, as now this is the first component to exceed a load of 1 at 850 new arrivals per second.

Based on this, for the remainder of this work, we consider an arrival rate of 840 the highest load without running into overload and an arrival rate of 850 the lowest load that already leads to overload behavior. From there, we extrapolate the load applied to the system by increasing the arrival rate even further, up to 3400 arrivals per second, which results in an overload factor of four, meaning four times more devices attempt to attach to the system than can be processed by the components in any given time interval. As this extrapolation of load, however, does not affect the processing characteristics of the system, the obtained results can still be considered representative of the real-world system. In addition, the comparisons performed in the following sections always compare results obtained from the simulation model. Hence, any abstractions included in the model apply to all data points that are directly compared.

C. Dedicated Overload Mechanisms

As resource scaling is only feasible up to a certain point, we now demonstrate the capability to evaluate specific mechanisms regarding the modeled system by defining and implementing various overload control mechanisms that allow the system to keep working, even while under severe overload.

Using the information obtained earlier information, we can subject the system to various levels of overload by increasing the arrival rate of new devices beyond the identified rate of 850 devices per second. To this end, we first introduce a set of three different overload mechanisms and compare their performance regarding system as well as device parameters.

1) Simple Dropping: The first mechanism employed and examined here, *Simple Dropping* aims at reducing the total waiting time of messages and thereby reducing the probability for timeouts that in return lead to a retry by the device. To this end, we limit the maximum queue size of each component in order to drop messages that can not be processed in time, thereby limiting the amount of unnecessary work performed by the system.

Fig. 9 shows the attachment rate as well as queue size for the AUC and CMAP components over time for an arrival rate of 850 new devices per second. Note that the queue size of other components is omitted here, since it remains zero over the whole duration. The attachment rate in Fig. 9(a) simply represents the total number of devices that are able to successfully complete their attachment cycle per second. The colors indicate the respective configuration of the maximum queue size, with *Inf* indicating the baseline case with an unlimited queue size. It can be seen that, for the baseline scenario, the rate stays largely linear around the 840 devices per second mark, until the queues fill up to a certain threshold, after which the aforementioned retry livelock occurs and the attachment rate quickly declines to zero. When looking at the baseline curve in Fig. 9(b), it can be seen that at the point of the decline of the attachment rate, the queue size of the AUC component exhibits exponential growth, as devices start to retry in addition to the arrival of new devices. Eventually, close to the 300-s mark, the load on the system even exceeds the capacity of the CMAP component and its queue starts to fill up as well.

Moving to the simple dropping scenarios shown in red, green and blue, it can be seen that the limitation of the maximum queue size has no positive effect on the survivability of the system. In fact, the opposite occurs, and the system enters the retry livelock even sooner, depending on the configured maximum queue size. This can be explained by the fact that, in order for devices to not issue retries, it is required to successfully process the four previously mentioned signaling messages. However, as the system is overloaded, the maximum configured queue size is eventually reached and a significant fraction of messages gets dropped. Hence, the probability of four consecutive messages of a single device not being dropped is close to zero, leading to the exact same behavior observed in the baseline scenario.

2) MSU Policing: The logical evolution to the simple dropping mechanism, *MSU policing*, limits the number of messages

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.

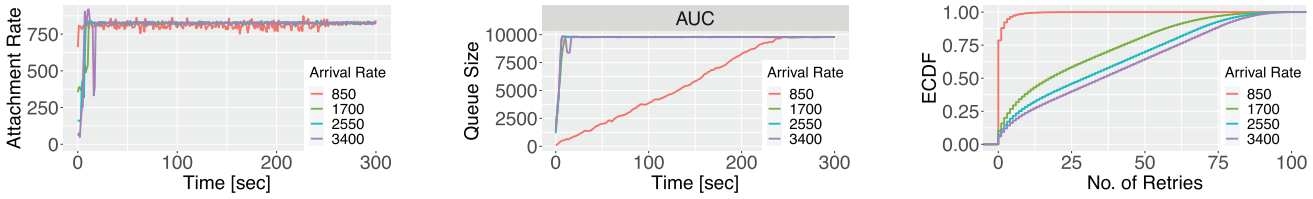


Fig. 10. Performance metrics over time when applying MSU policing. (a) Attachment rate. (b) Queue size for AUC component. (c) ECDF of retry count per device.

that are allowed to be processed concurrently. Thereby, each message type m is subject to its own quota Q_m, meaning that at one point in time, a maximum of $Q_{\text{SAI}_{\text{VLR}}}$ messages may be processed, while at the same time a maximum number of Q_{UL} may be in the system as well. Note that "being processed" here also includes messages currently waiting at one of the core components. This mechanism allows the system to perform a form of prioritized dropping, as it can reject messages of new devices, while still processing messages of devices that have already started their attachment cycle in time.

Fig. 10(a) and (b) again shows the attachment rate as well as queue size. Note that the queue size for the CMAP component is omitted here, since it remains zero over the simulated duration. The color now indicates the arrival rate of new devices, as we increase the arrival rate beyond the identified overload threshold of 850 devices per second to stress the system even further. Fig. 10(a) shows that the attachment rate, after an initial transient phase, remains constant at around 840 devices per second, even after the queue of the AUC component has filled [cf. Fig. 10(b)]. The quotas Q_m for all message types have been set to 5000 messages in this scenario, as a queue of 5000 SAI_{VLR} and 5000 SAI_{SGSN} at the AUC component allows the system to still process all waiting messages in time. In combination with the prioritized dropping, the UL and UL_GPRS messages of devices can also be processed in time. However, additional authentication messages beyond the threshold of $Q_{\text{SAI}_{\text{VLR}}} = 5000$ get rejected. This leads to near optimal resource utilization, as all messages processed by the system eventually lead to a successful attachment. Other combinations of the Q_m quotas have been evaluated as well and have resulted in similar behavior, which is why only one parameter set is shown here.

Additionally, Fig. 10(c) shows the ECDF of the number of retries devices have to perform before being accepted for processing. Intuitively, the number of retries grows with the arrival rate. The highest observed retry count was 100.

3) Device Policing With Explicit Congestion Signaling: Similar to the MSU policing mechanism, device policing also artificially limits the number of messages allowed to be concurrently present in the system. However, instead of tracking the number of messages, the mechanism keeps track of the total number of devices having at least one message in the system. Hence, the quota Q_D dictates the number of devices allowed to have messages currently being processed by the system. Note that this is a special interpretation of the MSU Policing mechanism in which all quotas but $Q_{\text{SAI}_{\text{VLR}}}$ are set to zero. This results in devices only being blocked on the first message of their attachment cycle, while all subsequent messages are never (intentionally) dropped.

In addition, we now introduce a mechanism that is able to explicitly notify devices about system congestion and can suggest a retry window based on currently unfinished work. Note that a mechanism like this would require an extension of the protocol stack to support the explicit congestion notification. Here, whenever Q_D is exceeded, the system rejects all messages of devices not currently being processed and notifies them about a point in the future at which a retry should be attempted. By this, the system gains more fine-grained control over the arrival rate of messages and can optimize resource utilization without excessive blocking of messages. Note that this scenario introduces changes to the device behavior model introduced earlier. Instead of pure timeout-driven retries, devices are now able to dynamically change their retry behavior. Also note that a mechanism like this would require an extension of current protocols and assumes devices that respect the feedback of the system. Especially the latter is unlikely to hold true in the real world. However, we still include this mechanism as a potential solution that optimizes both system utilization and message reject rate, thereby reducing the load on both the visited network and the signaling carrier in a roaming scenario. Furthermore, this can lead to reduced energy consumption for the IoT devices as excessive retries are reduced.

In the scenario presented here, the system is configured to propose retry times based on the fact that it can process 840 devices per second without resulting in overload. Hence, the timer t_r suggested to devices is computed as

$$t_r = \left\lceil \frac{n_u}{840} \right\rceil$$

with n_u being the number of unattached devices actively trying to connect to the system. This results in an arrival rate of exactly 840 devices per second when only taking into account retrying devices. In addition, the system has to deal with new devices. However, as the source for the overload is expected to be resolved eventually, and we want to ensure maximum resource utilization, a guaranteed arrival rate of 840 devices will yield the best resource utilization as it guarantees a high system load without resulting in overload.

Fig. 11 again shows the attachment rate as well as the ECDF of the number of retries for different arrival rates of new devices. It can be seen, that the attachment rate is as consistent as in the MSU policing scenario, while the number

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.

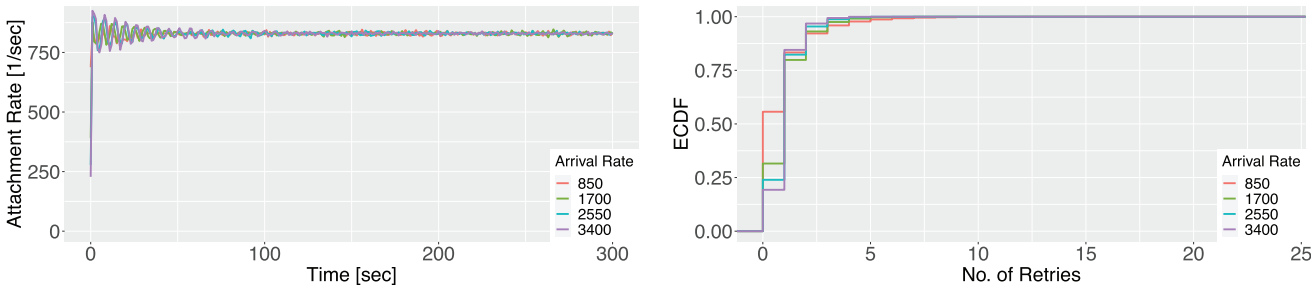


Fig. 11. Performance metrics over time when applying device policing with explicit signaling. (a) Attachment rate. (b) ECDF of retry count per device.

of required retries is significantly reduced, as the maximum observed value has declined from 100 to now 25 with the 99% quantile being only 3 in the case of 3400 arrivals per second. However, the time to attach, meaning the time between the first message attempt and the successful completion, remained similar with a mean time to attach of 236 s for MSU policing and 229 s for device policing with explicit congestion signaling.

VIII. Conclusion

The rapid adoption of IoT and the corresponding growth in number of devices has given rise to specialized platforms providing mobile connectivity for IoT use cases. Such a platform has been modeled in detail in this work. The developed simulation describes the fully virtualized mobile core of a real MVNO providing global IoT connectivity through roaming within 180 countries. In such a system, relying on globally distributed components located in the visited networks of devices as well as signaling carriers to transport messages between visited and home networks, short-term overload, incidents, and outages are expected to happen and need to be resolved while avoiding availability issues.

In order to improve our understanding of the behavior of such mobile core systems, we have presented a protocol-level simulation framework of a real-world 2G/3G MVNO mobile core and have shown that the proposed simulation is able to accurately reproduce several KPIs of the underlying real-world system. We focus on these generations of mobile communication as they are predominantly used by IoT devices in the wild [1]. The implementation used in this work is available on GitHub and can easily be extended to include more complex signaling models or investigate other aspects beside the ones examined in this work. Additionally, we thereby lay the basis for similar investigations in future 5G networks by developing models that can be applied to other technologies beyond the 2G/3G environment evaluated in this work. Obviously, the specific signaling procedures differ between mobile generations. However, the underlying device behavior is largely independent, and the resulting signaling model can easily be adapted and applied using our methodology.

In order to investigate and improve the processing capabilities of the modeled mobile core and to ensure the survival and recovery during overload phases, we investigated different overload mechanisms. We have shown the source of the issue in overload scenarios within the retry behavior of devices and proposed two different solutions to ensure recovery. Both MSU policing and device policing have been shown to ensure high resource utilization by allowing the system to operate at maximum capacity, even under sustained overload. While the prioritized dropping of the MSU policing mechanism allows the system to continue efficient operation, the extension of explicit congestion signaling introduced for the device policing mechanism may be leveraged to reduce the load on both the visited network components and signaling carriers as the explicit retry notification significantly reduces the number of unsuccessful retries while keeping a similar time to attach.

By providing our simulation tool, we enable the community to perform parameter studies and investigations that are unfeasible to perform in real-world systems and provide a platform for future research regarding the interaction between globally distributed IoT devices and centralized MVNO core networks.

We plan to extend the signaling model by taking into account different device classes with distinct signaling behavior, as previous studies have confirmed significant differences regarding the signaling behavior of IoT devices [6], such as random or periodic intervals between signaling interactions.

Acknowledgment

The authors alone are responsible for the content.

References

[1] A. Lutu, B. Jun, A. Finamore, F. Bustamante, and D. Perino, "Where things roam: Uncovering cellular IoT/M2M connectivity," 2020, *arXiv:2007.13708*.

[2] A. Nordrum et al., "Popular Internet of Things forecast of 50 billion devices by 2020 is outdated," *IEEE Spectr.*, vol. 18, no. 3, Aug. 2016. [Online]. Available: https://spectrum.ieee.org/popular-internet-of-things-forecast-of-50-billion-devices-by-2020-is-outdated

[3] Global Mobile Data Traffic Forecast, "Cisco visual networking index: Global mobile data traffic forecast update, 2017–2022," Cisco, San Jose, CA, USA, White Paper, 2019.

[4] P. Cerwall et al., *Ericsson Mobility Report, on the Pulse of the Networked Society. Hg. v. Ericsson*, Ericsson, Stockholm, Sweden, 2017.

[5] A. Čolaković and M. Hadžialić, "Internet of Things (IoT): A review of enabling technologies, challenges, and open research issues," *Comput. Netw.*, vol. 144, pp. 17–39, Oct. 2018.

[6] S. Geissler, F. Wamser, W. Bauer, M. Krolikowski, S. Gebert, and T. Hoßfeld, "Signaling traffic in Internet-of-Things mobile networks," in *Proc. IFIP/IEEE Int. Symp. Integr. Netw. Manag. (IM)*, May 2021, pp. 452–458.

[7] R. Samoilenko, N. Accurso, and F. Malandra, "A simulation study on the impact of IoT traffic in a smart-city LTE network," in *Proc. IEEE 31st Annu. Int. Symp. Pers. Indoor Mobile Radio Commun.*, 2020, pp. 1–6.

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.

[8] V. Vomhoff, S. Geissler, F. Loh, W. Bauer, and T. Hossfeld, "Characterizing mobile signaling anomalies in the Internet-of-Things," in *Proc. IEEE/IFIP Netw. Oper. Manag. Symp.*, 2022, pp. 1–6.

[9] M. Singh, E. Fuenmayor, E. P. Hinchy, Y. Qiao, N. Murray, and D. Devine, "Digital twin: Origin to future," *Appl. Syst. Innov.*, vol. 4, no. 2, p. 36, 2021.

[10] H. Tahaei, F. Afifi, A. Asemi, F. Zaki, and N. B. Anuar, "The rise of traffic classification in IoT networks: A survey," *J. Netw. Comput. Appl.*, vol. 154, Mar. 2020, Art. no. 102538.

[11] F. Metzger, T. Hoßfeld, A. Bauer, S. Kounev, and P. E. Heegaard, "Modeling of aggregated IoT traffic and its application to an IoT cloud," *Proc. IEEE*, vol. 107, no. 4, pp. 679–694, Apr. 2019.

[12] A. Sivanathan et al., "Characterizing and classifying IoT traffic in smart cities and campuses," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2017, pp. 559–564.

[13] A. Sivanathan et al., "Classifying IoT devices in smart environments using network traffic characteristics," *IEEE Trans. Mobile Comput.*, vol. 18, no. 8, pp. 1745–1759, Aug. 2019.

[14] M. Laner, P. Svoboda, N. Nikaein, and M. Rupp, "Traffic models for machine type communications," in *Proc. 10th Int. Symp. Wireless Commun. Syst.*, 2013, pp. 1–5.

[15] A. Lutu, B. Jun, F. E. Bustamante, D. Perino, M. Bagnulo, and C. G. Bontje, "A first look at the IP eXchange ecosystem," *SIGCOMM Comput. Commun. Rev.*, vol. 50, no. 4, pp. 25–34, Oct. 2020.

[16] Y. Choi, C.-H. Yoon, Y.-S. Kim, S. W. Heo, and J. A. Silvester, "The impact of application signaling traffic on public land mobile networks," *IEEE Commun. Mag.*, vol. 52, no. 1, pp. 166–172, Jan. 2014.

[17] I. L. Bedhiaf, O. Cherkaoui, and G. Pujolle, "Third-generation virtualized architecture for the MVNO context," *Ann. Telecommun.*, vol. 64, no. 5, pp. 339–347, 2009.

[18] Y. Li et al., "Understanding the ecosystem and addressing the fundamental concerns of commercial MVNO," *IEEE/ACM Trans. Netw.*, vol. 28, no. 3, pp. 1364–1377, Jun. 2020.

[19] D. B. Johnson, "Validation of wireless and mobile network models and simulation," in *Proc. DARPA/NIST Netw. Simul. Validation Workshop*, 1999, pp. 1–3.

[20] A. P. Jardosh, E. M. Belding-Royer, K. C. Almeroth, and S. Suri, "Real-world environment models for mobile network evaluation," *IEEE J. Sel. Areas Commun.*, vol. 23, no. 3, pp. 622–632, Mar. 2005.

[21] F. Metzger, C. Schwartz, and T. Hoßfeld, "GTP-based load model and virtualization gain for a mobile network's GGSN," in *Proc. IEEE 5th Int. Conf. Commun. Electron. (ICCE)*, 2014, pp. 206–211.

[22] M. H. Mazhar and Z. Shafiq, "Characterizing smart home IoT traffic in the wild," in *Proc. IEEE/ACM 5th Int. Conf. Internet Things Des. Implement. (IoTDI)*, 2020, pp. 203–215.

[23] N. Xia, H.-H. Chen, and C.-S. Yang, "Radio resource management in machine-to-machine communications—A survey," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 1, pp. 791–828, 1st Quart., 2018.

[24] S. Duan, V. Shah-Mansouri, Z. Wang, and V. W. S. Wong, "D-ACB: Adaptive congestion control algorithm for bursty M2M traffic in LTE networks," *IEEE Trans. Veh. Technol.*, vol. 65, no. 12, pp. 9847–9861, Dec. 2016.

[25] N. Jiang, Y. Deng, A. Nallanathan, X. Kang, and T. Q. S. Quek, "Analyzing random access collisions in massive IoT networks," *IEEE Trans. Wireless Commun.*, vol. 17, no. 10, pp. 6853–6870, Oct. 2018.

[26] R. Ratasuk, N. Mangalvedhe, and A. Ghosh, "Overview of LTE enhancements for cellular IoT," in *Proc. IEEE 26th Annu. Int. Symp. Pers. Indoor Mobile Radio Commun. (PIMRC)*, 2015, pp. 2293–2297.

[27] M.-Y. Cheng, G.-Y. Lin, H.-Y. Wei, and A. C.-C. Hsu, "Overload control for machine-type-communications in LTE-advanced system," *IEEE Commun. Mag.*, vol. 50, no. 6, pp. 38–45, Jun. 2012.

[28] A. Ksentini, Y. Hadjadj-Aoul, and T. Taleb, "Cellular-based machine-to-machine: Overload control," *IEEE Netw.*, vol. 26, no. 6, pp. 54–60, Nov./Dec. 2012.

[29] M. Gotin, D. Werle, F. Lösch, A. Koziolek, and R. Reussner, "Overload protection of cloud-IoT applications by feedback control of smart devices," in *Proc. ACM/SPEC Int. Conf. Perform. Eng.*, 2019, pp. 51–58.

[30] L. Ferdouse, A. Anpalagan, and S. Misra, "Congestion and overload control techniques in massive M2M systems: A survey," *Trans. Emerg. Telecommun. Technol.*, vol. 28, no. 2, p. e2936, 2017.

[31] "System improvements for machine-type communications," 3GPP, Sophia Antipolis, France, 3GPP Rep. TR 23.888, 2012.

[32] A. Larmo and R. Susitaival, "RAN overload control for machine type communications in LTE," in *Proc. IEEE Globecom Workshops*, 2012, pp. 1626–1631.

[33] M. Alvi, K. M. Abualnaja, W. T. Toor, and M. Saadi, "Performance analysis of access class barring for next generation IoT devices," *Alexandria Eng. J.*, vol. 60, no. 1, pp. 615–627, 2021.

[34] Y.-H. Hsu, K. Wang, and Y.-C. Tseng, "Enhanced cooperative access class barring and traffic adaptive radio resource management for M2M communications over LTE-A," in *Proc. Asia–Pacific Signal Inf. Process. Assoc. Annu. Summit Conf.*, 2013, pp. 1–6.

[35] S.-Y. Lien, T.-H. Liau, C.-Y. Kao, and K.-C. Chen, "Cooperative access class barring for machine-to-machine communications," *IEEE Trans. Wireless Commun.*, vol. 11, no. 1, pp. 27–32, Jan. 2012.

[36] S.-Y. Lien, K.-C. Chen, and Y. Lin, "Toward ubiquitous massive accesses in 3GPP machine-to-machine communications," *IEEE Commun. Mag.*, vol. 49, no. 4, pp. 66–74, Apr. 2011.

[37] C.-Y. Tu, C.-Y. Ho, and C.-Y. Huang, "Energy-efficient algorithms and evaluations for massive access management in cellular based machine to machine communications," in *Proc. IEEE Veh. Technol. Conf. (VTC-Fall)*, 2011, pp. 1–5.

[38] T. Taleb and A. Kunz, "Machine type communications in 3GPP networks: Potential, challenges, and solutions," *IEEE Commun. Mag.*, vol. 50, no. 3, pp. 178–184, Mar. 2012.

[39] T. Taleb and A. Ksentini, "An efficient scheme for MTC overload control based on signaling message compression," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2013, pp. 342–346.

[40] A. Amokrane, A. Ksentini, Y. Hadjadj-Aoul, and T. Taleb, "Congestion control for machine type communications," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2012, pp. 778–782.

[41] D. Gannon, R. Barga, and N. Sundaresan, "Cloud-native applications," *IEEE Cloud Comput.*, vol. 4, no. 5, pp. 16–21, Sep./Oct. 2017.

[42] N. Kratzke and P.-C. Quint, "Understanding cloud-native applications after 10 years of cloud computing—A systematic mapping study," *J. Syst. Softw.*, vol. 126, pp. 1–16, Apr. 2017.

[43] M. Giess and D. Sisalem, "Multi-layered distributed GTP-C processing," E.U. Patent EP3 806 412, Apr. 14, 2021.

[44] I. Baldini et al., "Serverless computing: Current trends and open problems," in *Research Advances in Cloud Computing*. Singapore: Springer, 2017, pp. 1–20.

[45] R. J. Byrd, S. E. Smith, and S. P. deJong, "An actor-based programming system," in *Proc. SIGOA Conf. Office Inf. Syst.*, 1982, pp. 67–78.

[46] P. Haller and M. Odersky, "Scala actors: Unifying thread-based and event-based programming," *Theor. Comput. Sci.*, vol. 410, nos. 2–3, pp. 202–220, 2009.

[47] Akka. "Akka—Build concurrent, distributed, and resilient message-driven applications for Java and scala." Lightbend, Inc. Apr. 2023. [Online]. Available: https://akka.io/

[48] *Digital Cellular Telecommunications System (Phase 2+); Universal Mobile Telecommunications System (UMTS); Mobile Application Part (MAP) Specification (3GPP TS 29.002 Version 10.2.0 Release 10)*, ETSI Standard TS 129 002 V10. 2.0, Apr. 2011.



Stefan Geißler received the Ph.D. degree for his thesis "Performance Evaluation of Next-Generation Data Plane Architectures and their Components" from the University of WÜrzburg, WÜrzburg, Germany, in 2022..

He is a Research Associate with the Chair of Communication Networks, University of Würzburg, where he leads the Cloud Applications and Networks Research Group. His current research is focused on the analytical and simulative performance evaluation and optimization of next-generation communication technologies, including TSN, 5G, and Internet of Things.



Florian Wamser received the Ph.D. degree from the University of Würzburg, Würzburg, Germany, in 2015.

He is a Researcher and a Lecturer of Communication Networks and Cybersecurity with Lucerne University of Applied Sciences and Arts, Luzern, Switzerland, with an affinity for monitoring, data science and data engineering. He studied with the University of Würzburg and Helsinki University of Technology, Espoo, Finland. His current research is focused on the analytical and simulative performance evaluation and optimization of cloud and communication networks and related fields.

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.



Wolfgang Bauer received the Master Diploma in physics from the University of Würzburg, Würzburg, Germany.

He is working as a Core Network Engineer with EMnify GmbH, a German MVNO, Berlin, Germany. He is building and operating core network application, such as HLR and packet gateways in the cloud. In a joint research project with the University of Würzburg, he is working on strategies on how to improve resilience and scalability of mobile core networks.



Steffen Gebert received the Ph.D. degree for his thesis on software-based networks from the University of Würzburg, Würzburg, Germany, in 2016.

He is responsible for the production infrastructure with EMnify, Berlin, Germany, a CPaaS/MVNO running purpose-built mobile core networks for the Internet of Things. Before joining EMnify in 2017, he was a Researcher with the University of Würzburg. He is the Project Lead of the 5CALE project, a joint research project together with the University of Würzburg.



Samuel Kounev (Member, IEEE) received the Ph.D. degree from the University of Darmstadt, Darmstadt, Germany, in 2005.

He is a Professor with the Chair of Software Engineering, University of Würzburg, Würzburg, Germany. He recently coauthored the first textbook focused on systems benchmarking. His research is inspired by the vision of self-aware computing systems, to which he has been one of the major contributors shaping its development. His research interests span the areas of software design, modeling, and architecture-based analysis; systems benchmarking and experimental analysis; and autonomic computing.



Tobias Hoßfeld (Senior Member, IEEE) received the Ph.D. degree from the University of Würzburg, Würzburg, Germany, in 2014.

He has been a Professor with the Chair of Communication Networks, University of Würzburg, since 2018. From 2014 to 2018, he was the Head of the Chair "Modeling of Adaptive Systems," University of Duisburg–Essen, Duisburg, Germany. He recently coauthored a book on performance modeling and analysis of communication networks.

Prof. Hoßfeld received three awards for his Ph.D. Thesis and the Fred W. Ellersick Prize 2013 (IEEE Communications Society).

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:43:32 UTC from IEEE Xplore. Restrictions apply.